

**252-0027**

**Einführung in die Programmierung  
Übungen**

**Woche 4: Types, Schleifen, Methoden**

**Joran Bühler  
Departement Informatik  
ETH Zürich**

# Inhalt

- **JUnit/GitHub Information**
- **Schleifen**
- **Coding with Joran**
- **Strings**
- **Kahoot**
- **Vorbesprechung**

**JUnit**

# JUnit Tests: Introduction

- JUnit Test erlauben es Code zu testen.
- Jetzt: Tests benutzen
- Später: Tests selbst schreiben.

# JUnit Tests: IntelliJ

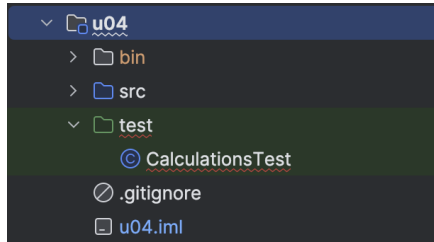
## ■ JUnit kann nicht gefunden werden?

© ArrayUtilTest.java ~/IdeaProjects/exercise-projects2025/u05/test 28 problems

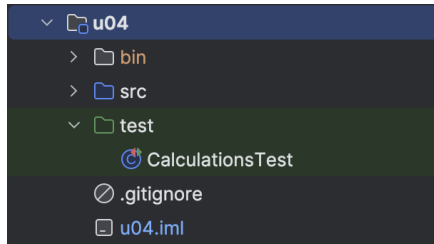
- ❗ Cannot resolve symbol 'Assertions' :1
- ❗ Cannot resolve symbol 'Assertions' :2
- ❗ Cannot resolve symbol 'Assertions' :3
- ❗ Cannot resolve symbol 'Test' :5
- ❗ Cannot resolve symbol 'Test' :9
- ❗ Cannot resolve method 'assertArrayEquals' in 'ArrayUtilTest' :11
- ❗ Cannot resolve method 'assertArrayEquals' in 'ArrayUtilTest' :13
- ❗ Cannot resolve method 'assertArrayEquals' in 'ArrayUtilTest' :14
- ❗ Cannot resolve method 'assertArrayEquals' in 'ArrayUtilTest' :16
- ❗ Cannot resolve method 'assertArrayEquals' in 'ArrayUtilTest' :17
- ❗ Cannot resolve method 'assertArrayEquals' in 'ArrayUtilTest' :18
- ❗ Cannot resolve method 'assertArrayEquals' in 'ArrayUtilTest' :19
- ❗ Cannot resolve method 'assertArrayEquals' in 'ArrayUtilTest' :20
- ❗ Cannot resolve method 'assertArrayEquals' in 'ArrayUtilTest' :21
- ❗ Cannot resolve method 'assertArrayEquals' in 'ArrayUtilTest' :22

# JUnit Tests: IntelliJ

- JUnit kann nicht gefunden werden?

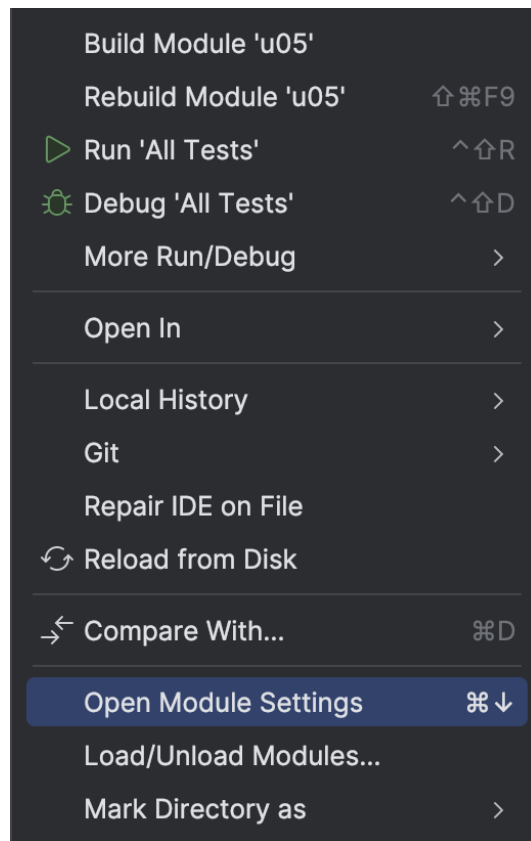


- JUnit library muss importiert werden!



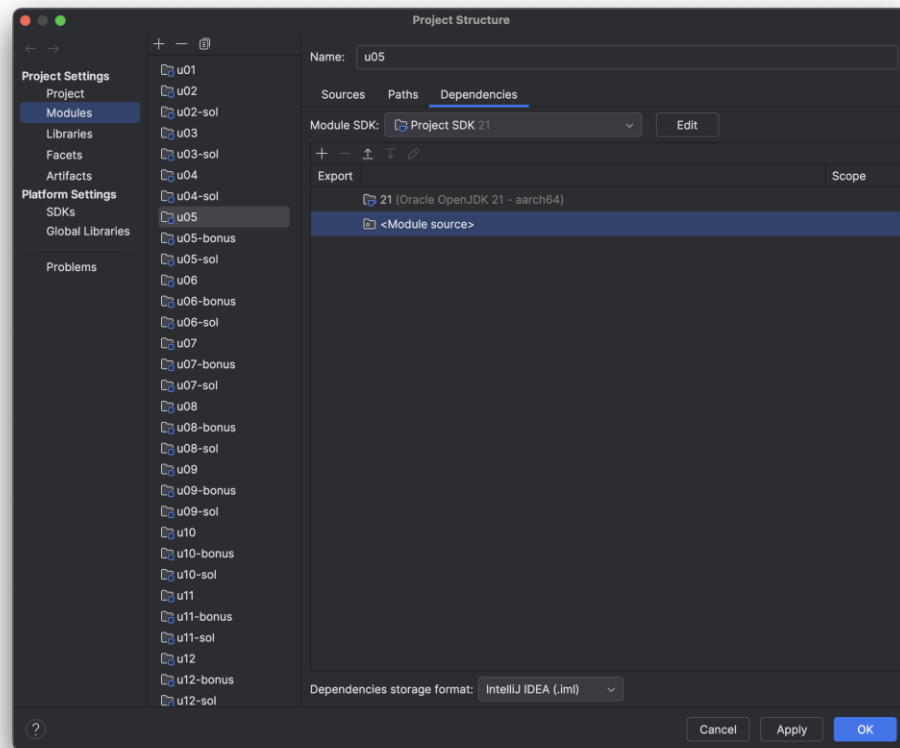
# JUnit Tests: JUnit Importieren

- **Rechtsklick auf Project Folder, z.B. u03.**
- **1. Open Module Settings**



# JUnit Tests: JUnit Importieren

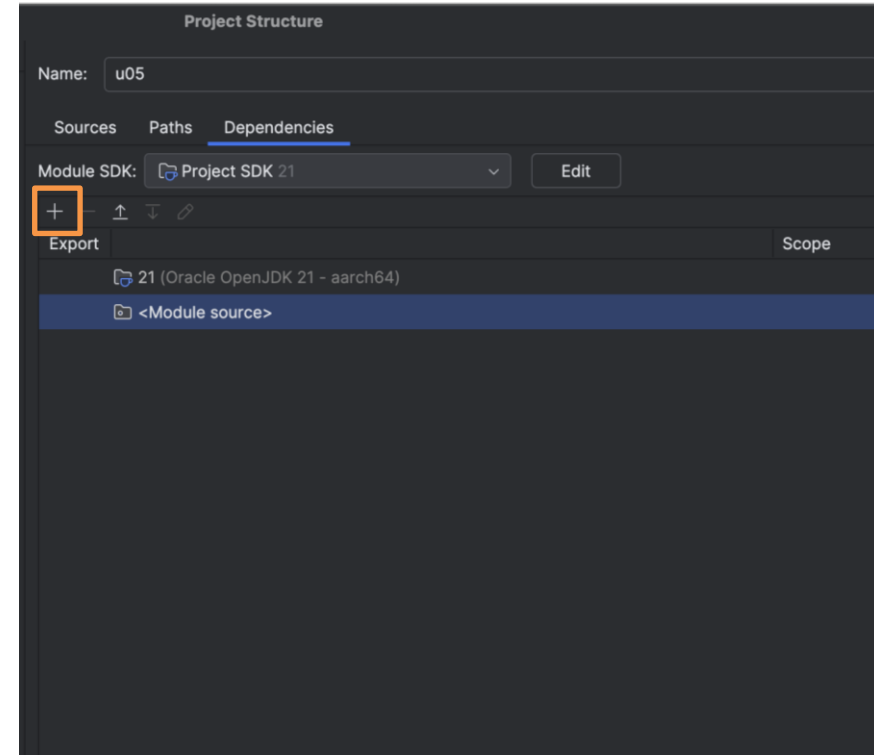
- **Rechtsklick auf Project Folder, z.B. u03.**
- **1. Open Module Settings**
- **2. Under Dependencies:**  
**Click +**





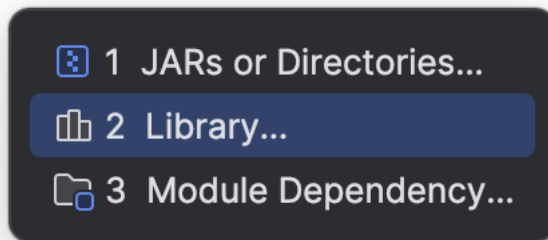
# JUnit Tests: JUnit Importieren

- Rechtsklick auf Project Folder, z.B. u03.
- 1. Open Module Settings
- 2. Under Dependencies: Click +



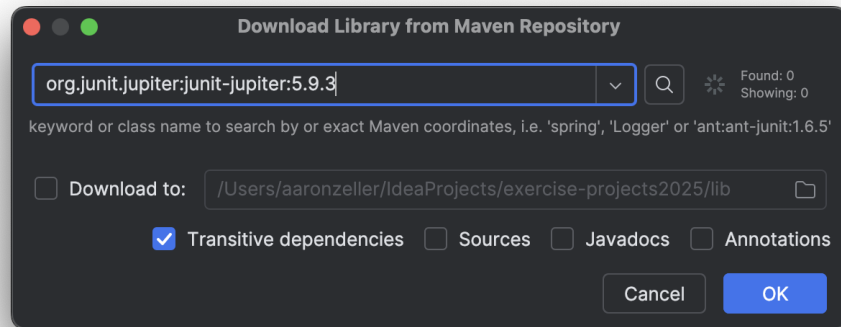
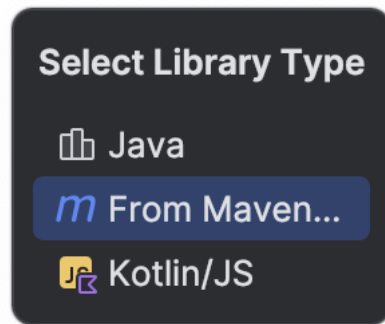
# JUnit Tests: JUnit Importieren

- Rechtsklick auf Project Folder, z.B. u03.
- 1. Open Module Settings
- 2. Under Dependencies: Click +
- 3. Library... -> New Library



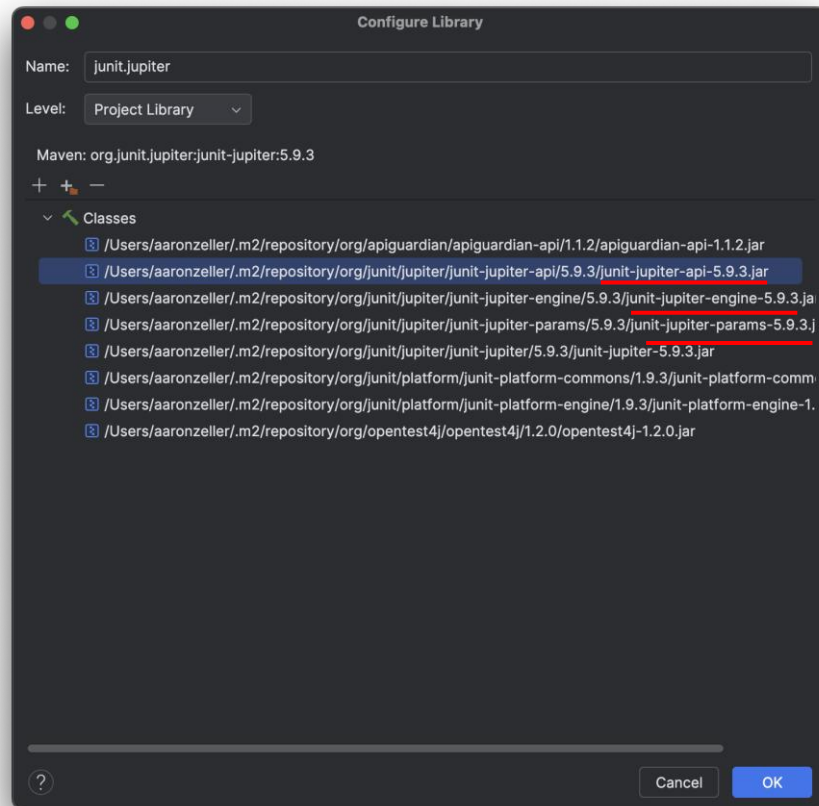
# JUnit Tests: JUnit Importieren

- Rechtsklick auf Project Folder, z.B. u03.
- 1. Open Module Settings
- 2. Wähle + in Project Structure
- 3. Library... -> New Library
- 4. From Maven... -> Suche org.junit.jupiter:junit-jupiter:5.9.3



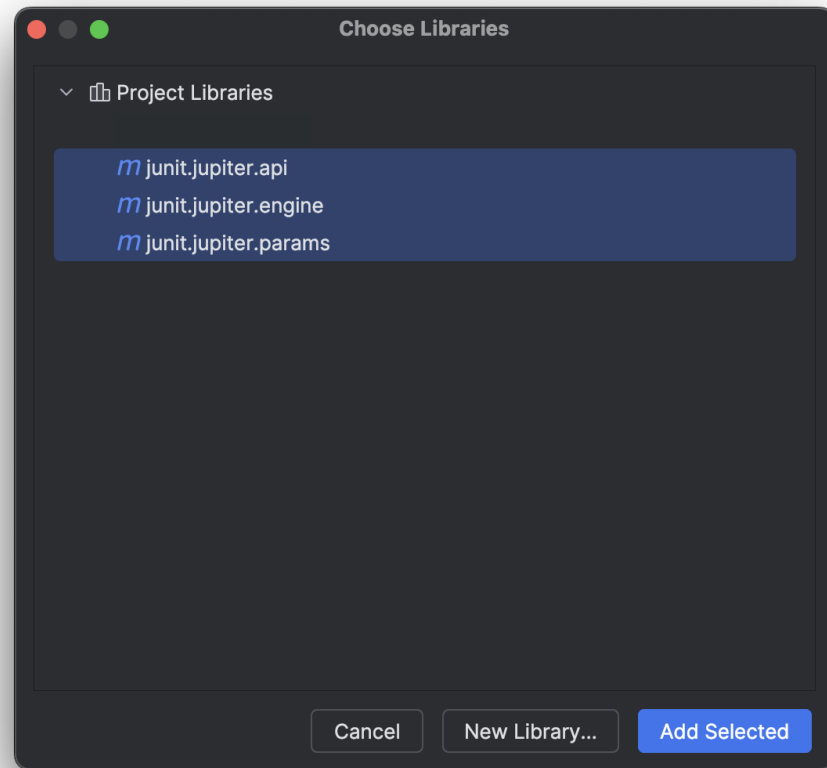
# JUnit Tests: JUnit Importieren

- **Wir brauchen:**
  - junit-jupiter-api-5.9.3.jar
  - junit-jupiter-engine.5.9.3.jar
  - junit-jupiter-params.5.9.3.jar
- **Wähle die jar-Dateien und klicke *OK***



# JUnit Tests: JUnit Importieren

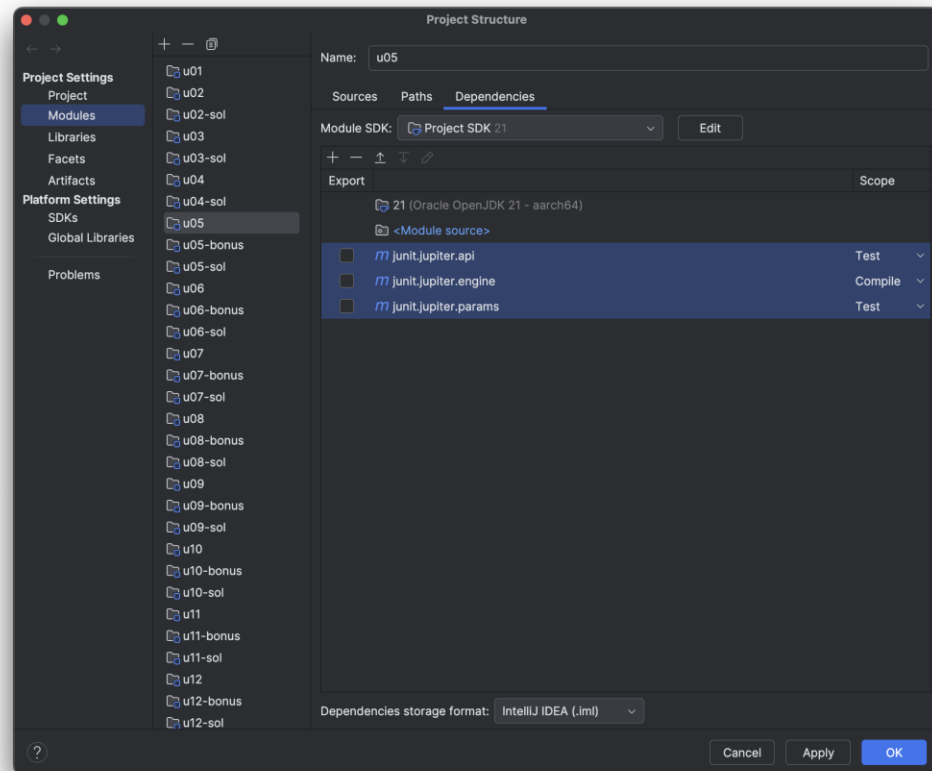
- **Wir brauchen:**
  - junit-jupiter-api-5.9.3.jar
  - junit-jupiter-engine.5.9.3.jar
  - junit-jupiter-params.5.9.3.jar
- **Wähle *Add Selected***



# JUnit Tests: JUnit Importieren

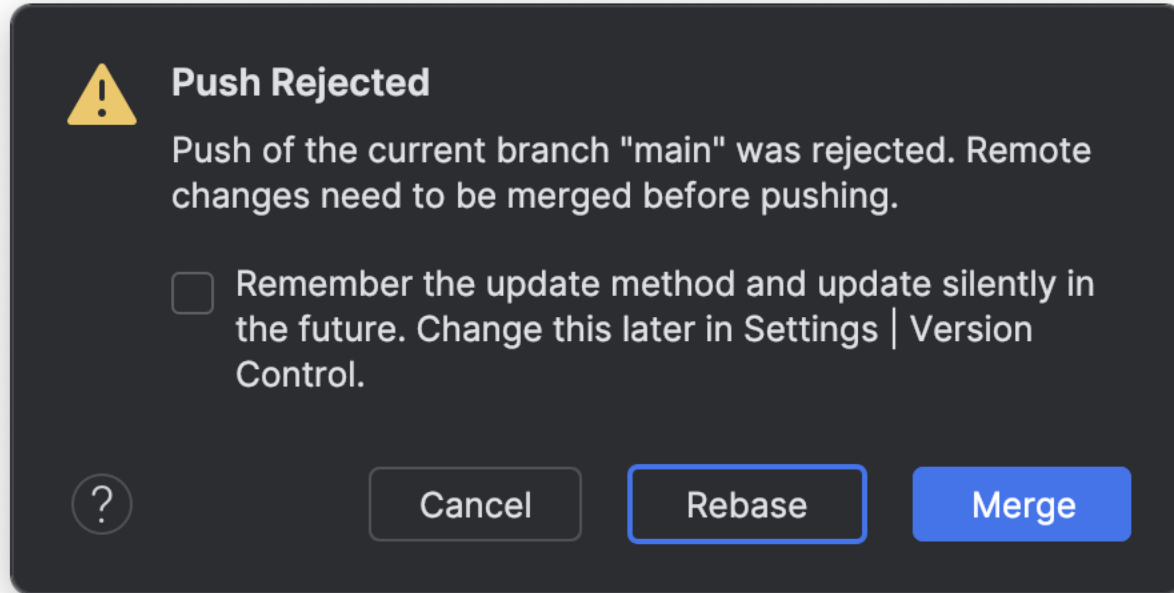
## ■ Wir brauchen:

- junit-jupiter-api-5.9.3.jar
- junit-jupiter-engine-5.9.3.jar
- junit-jupiter-params-5.9.3.jar



# **Git Merge Conflicts**

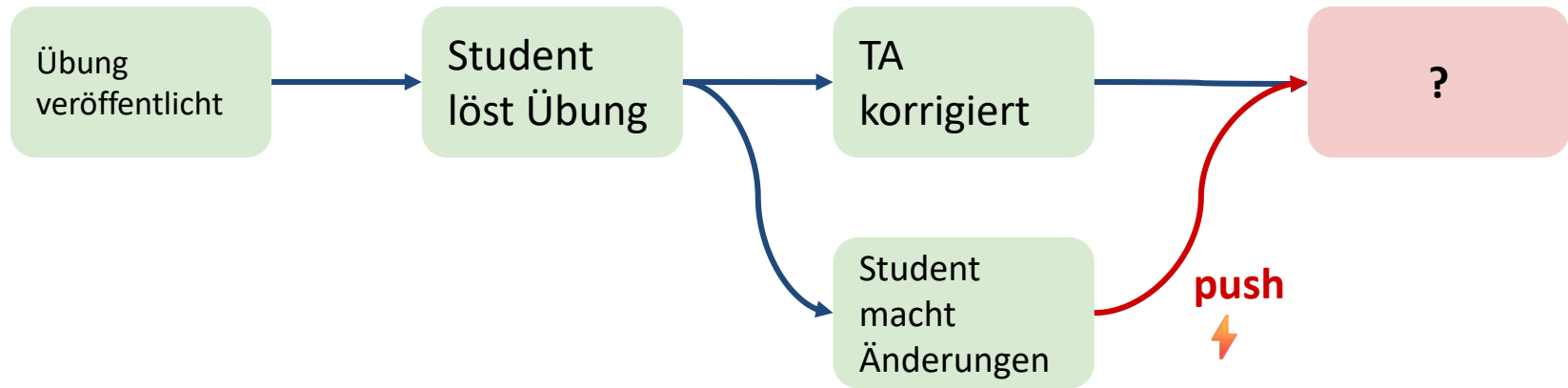
# Git sagt "Push Rejected"





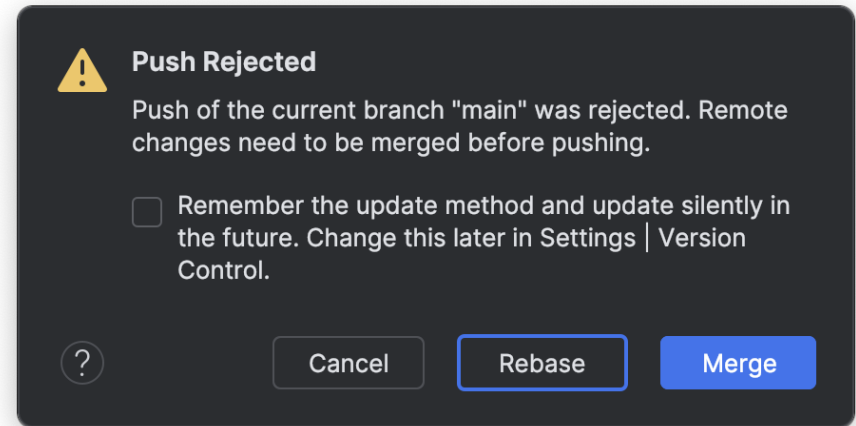
# Git - Merge Conflicts

Student löst Übung gleichzeitig während TA korrigiert  
→ Beim Versuch zu pushen, Konflikt!



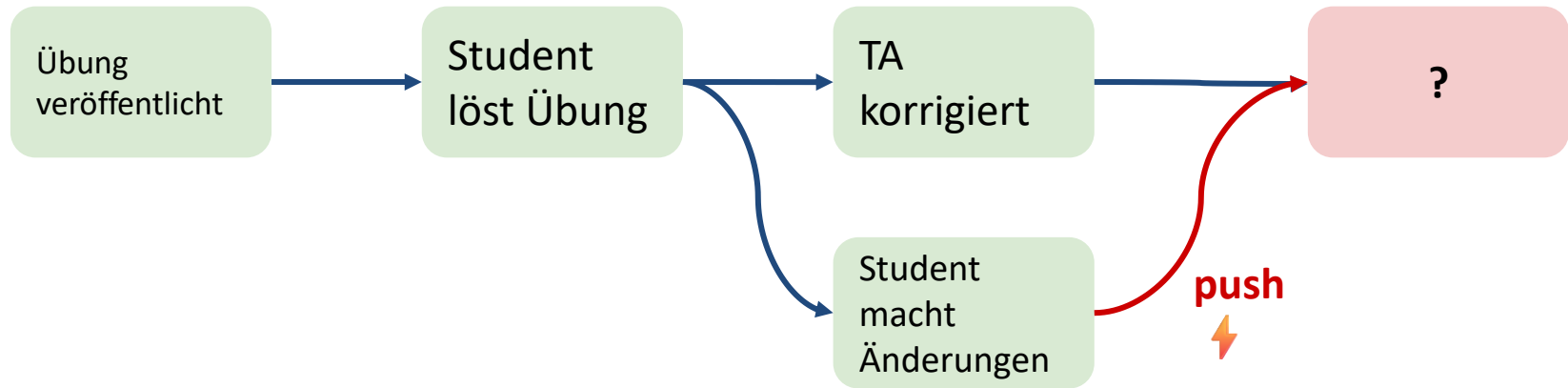
# Zwei Optionen

- 🔗 **Merge:** Kombiniert die zwei unterschiedlichen Versionen der Datei in eine kombinierte Datei.
- 🔗 **Rebase:** Übernimmt die Änderungen auf dem Server und versucht deine Änderungen anzuwenden.

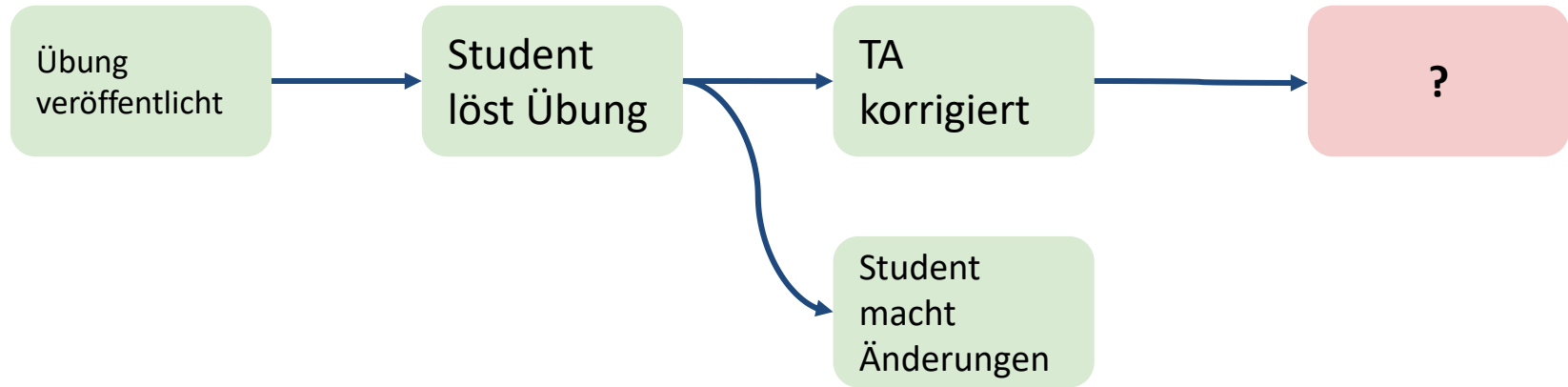


# Git Merge

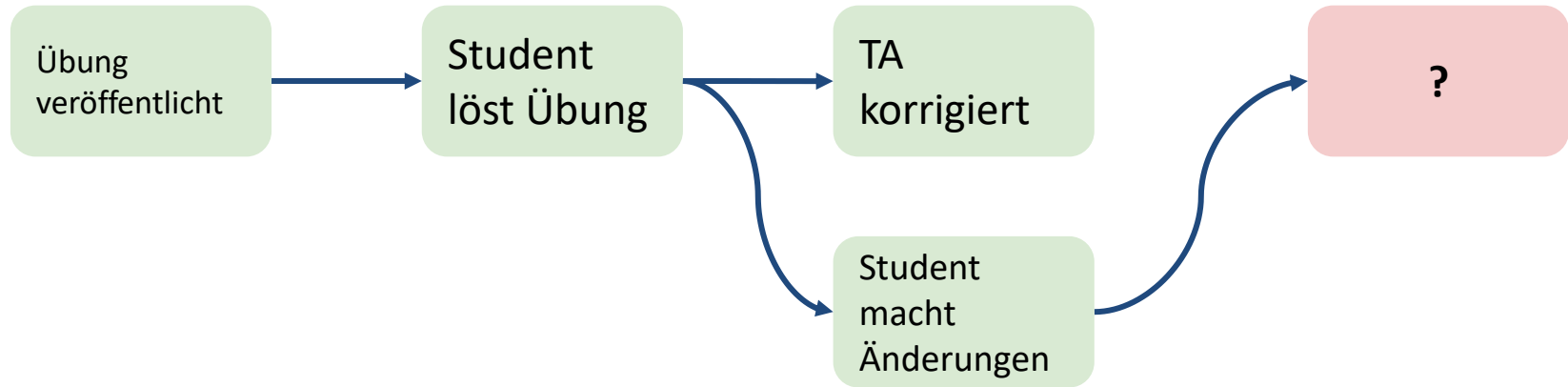
# Git – Merge



# Git – Merge (Accept Theirs)

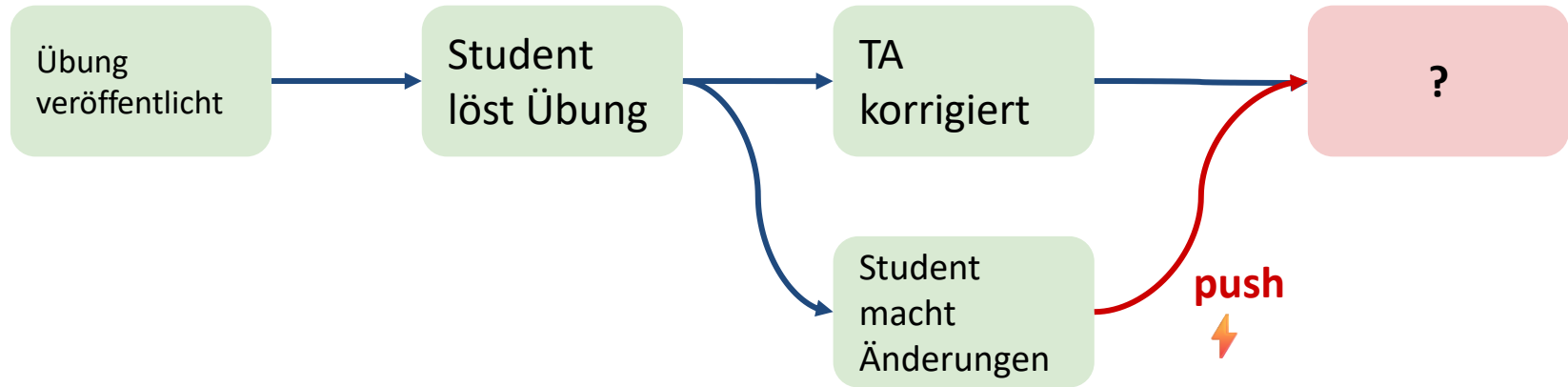


# Git – Merge (Accept Yours)



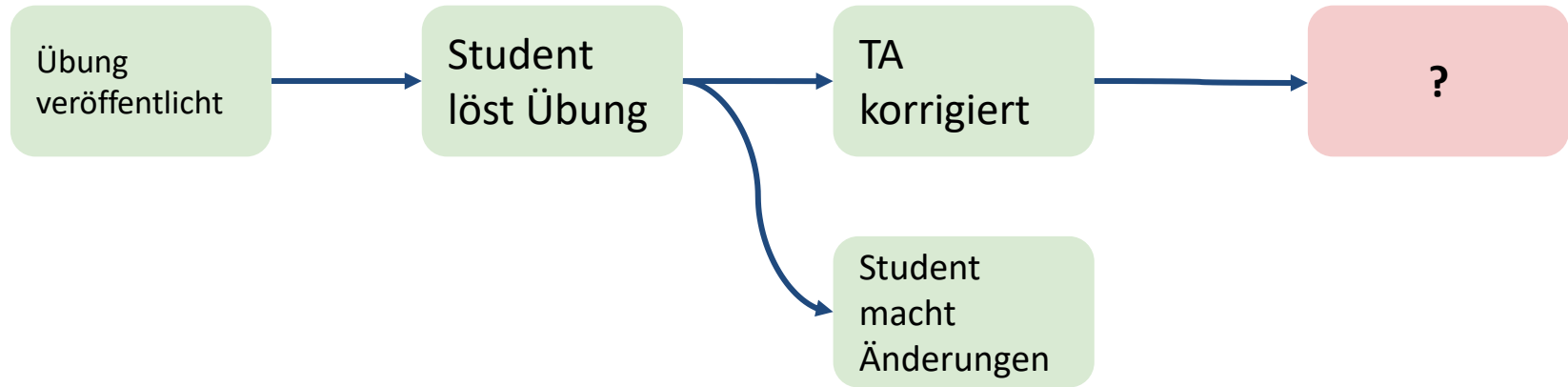
# Git Rebase

# Git – Rebase

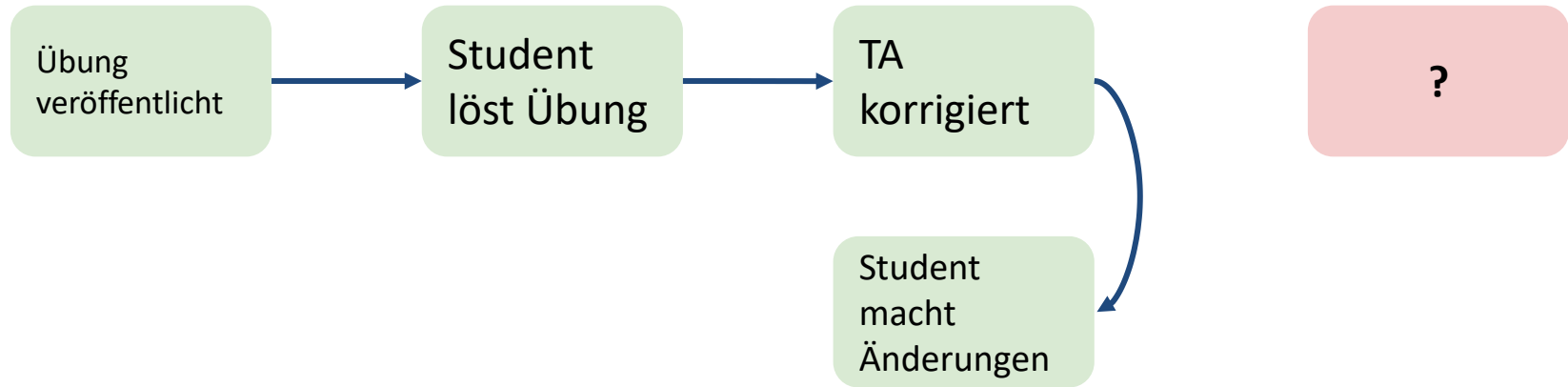




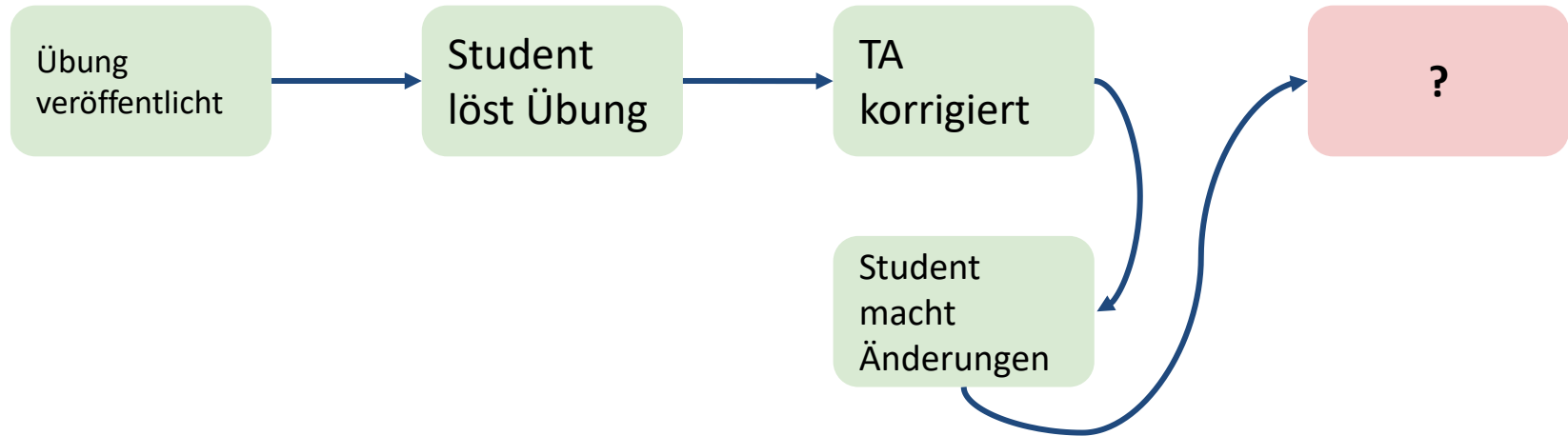
# Git – Rebase



# Git – Rebase



# Git – Rebase



# Zwei Situationen

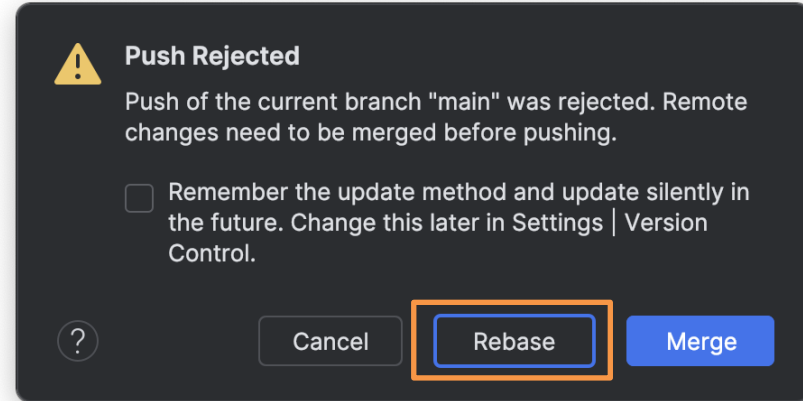
- ✂ TA und Student haben nicht dieselbe(n) Datei(en) bearbeitet. 🎉
- ✂ TA und Student haben dieselbe(n) Datei(en) bearbeitet. 😞

# Zwei Situationen

❧ TA und Student haben nicht dieselbe(n) Datei(en) bearbeitet. 🤖

❧ *Rebase* wählen.

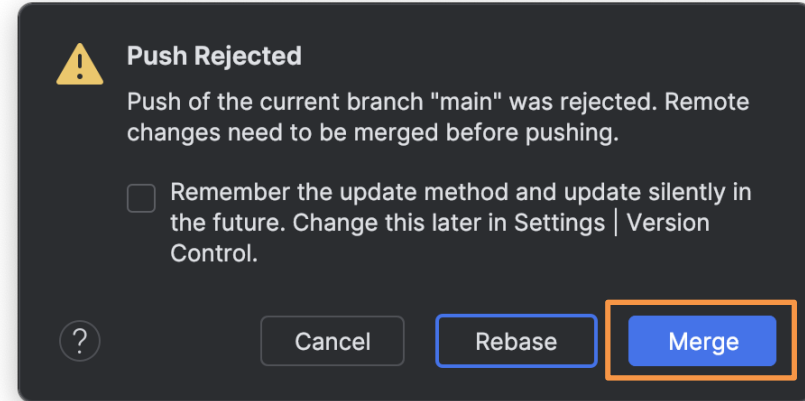
❧ Änderungen sind gepushed.



# Zwei Situationen

✂ TA und Student haben dieselbe(n) Datei(en) bearbeitet. 🤔

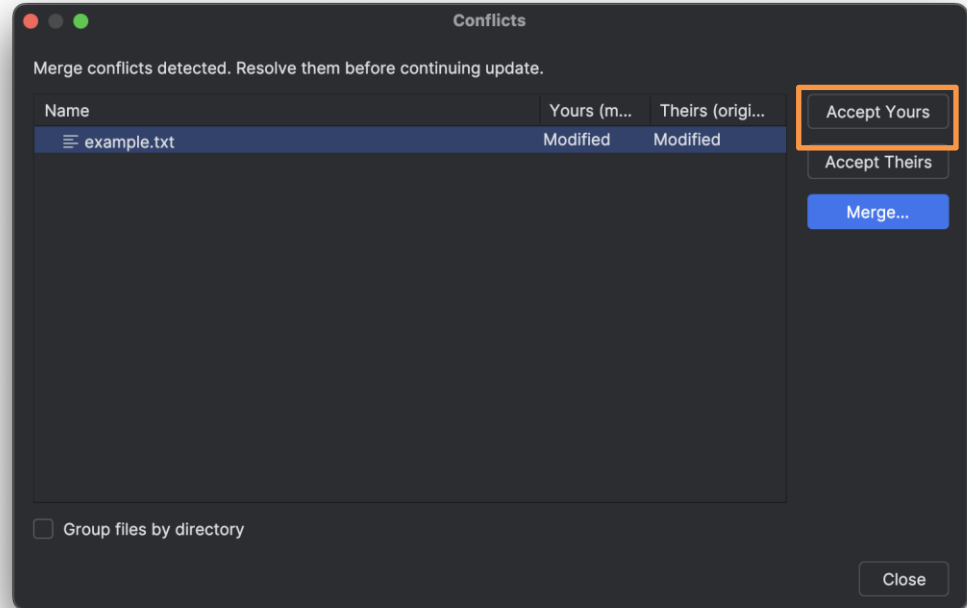
✂ *Merge* wählen.



# Zwei Situationen

✂ TA und Student haben dieselbe(n) Datei(en) bearbeitet. 🤔

- ✂ *Merge* wählen.
- ✂ *Accept Yours* wählen.
- ✂ Änderungen gepushed.



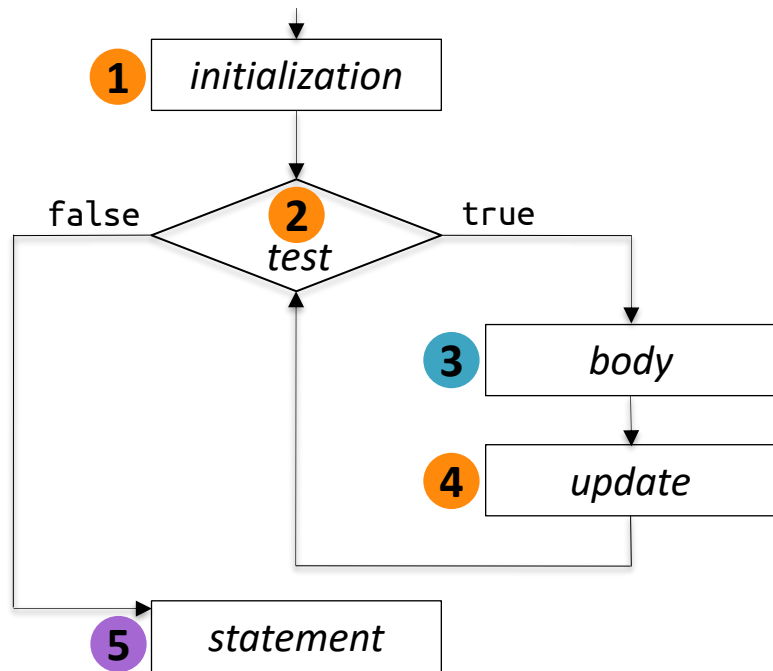
# Schleifen



# for-Schleife («for loop»): Kontrollfluss

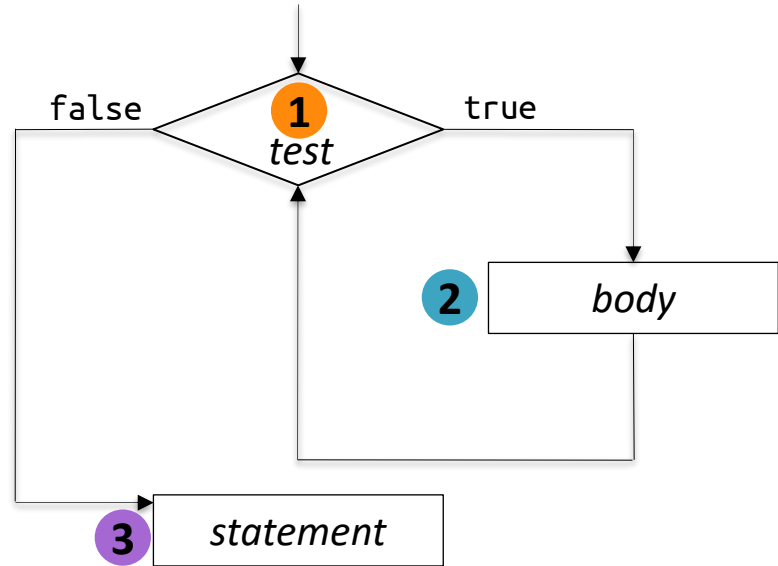
```
for (1initialization; 2test; 4update) {  
    3body  
}  
5statement;
```

```
for (int i = 1; i <= 5; i = i + 1) {  
    System.out.println(i); // 1 2 3 4 5  
}
```



# while-Schleife («while loop»)

```
while (test) {  
    body  
}  
statement;
```



while-Schleife führt Rumpf so lange aus, wie *test* den Wert *true* ergibt.  
In anderen Worten: die while-Schleife bricht ab, sobald *test* den Wert *false* ergibt.

# Schleifen

Was gibt diese Methode aus?

```
public static void main (String[] args) {  
    int f = 0; int g = 1;  
  
    for (int i = 0; i < 15; i++) {  
        System.out.print(" " + f);  
        f = f + g;  
        g = f - g;  
    }  
    System.out.println();  
}
```

| i   | f   | g   |
|-----|-----|-----|
|     | 0   | 1   |
| 0   | 0   | 1   |
| 0   | 0   | 1   |
| 0   | 1   | 1   |
| 0   | 1   | 0   |
| 1   | 1   | 0   |
| ... | ... | ... |

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

# Schleifen: While vs. For vs. Do-While

- **For-Loop:** Benutzen wir, wenn wir die Anzahl Iterationen bereits for der Ausführung des Loops kennen.
- **While:** Benutzen wir, wenn wir die Anzahl Iterationen nicht kennen und diese Abhängig von einer Bedingung ist.
- **Do-While:** Wird benutzt, wenn wir zuerst den Code (im Loop Body) ausführen und dann erst die Bedingung prüfen wollen.

# Schleifen: For vs. While

- While und Do-While können das gleiche.
- For und While aber auch!

# Schleifen: For vs. While – Beispiel 1



```
1 public class MyClass {  
2     public static void main(String[] args){  
3         for(i = 0; i < 5; i++){  
4             System.out.println("i = " + i);  
5         }  
6     }  
7 }
```

# Schleifen: For vs. While – Beispiel 1



```
1 public class MyClass {  
2     public static void main(String[] args){  
3         int i = 0;           // Initialisierung  
4         while(i < 5){       // Bedingung  
5             System.out.println("i = " + i);  
6             i++;             // Aktualisierung  
7         }  
8     }  
9 }
```

# Schleifen: For vs. While – Beispiel 2



```
1 public class MyClass {  
2     public static void main(String[] args){  
3         int i = 1;  
4         while(fancyCheck(i)){  
5             int result = complexOperation(i);  
6             i = i * 42 - 15;  
7             System.out.println(result);  
8         }  
9     }  
10 }
```



# Schleifen: For vs. While – Beispiel 2



```
1 public class MyClass {
2     public static void main(String[] args){
3         int i = 1;
4         for(; fancyCheck(i); ){
5             int result = complexOperation(i);
6             i = i * 42 - 15;
7             System.out.println(result);
8         }
9     }
10 }
```

# Schleifen: Do-While?

- Selten genutzt aber kann sehr nützlich sein!



```
1 public class MyClass {  
2     public static void main(String[] args){  
3         Scanner console = new Scanner(System.in);  
4         String pin;  
5         do {  
6             pin = console.nextLine();  
7         } while(!isPassword(pin));  
8         System.out.println("You logged in successfully!");  
9         console.close();  
10    }  
11 }
```

# **Inkrement und Dekrement**

# Inkrement und Dekrement

- **Präfix: ++i**

```
int a = 5;  
int b = ++a; // Zuerst wird a inkrementiert, dann wird a zugewiesen.  
// a = 6, b = 6
```

- **Postfix i++**

```
int a = 5;  
int b = a++; // Zuerst wird a zugewiesen, dann wird a inkrementiert.  
// a = 6, b = 5
```

Diese Operatoren sind besonders nützlich in Schleifen, um beispielsweise einen Zähler zu erhöhen oder zu verringern.

# Weitere Kurzformen

- **Prä-Inkrement** («pre-increment») und **Prä-Dekrement** («pre-decrement»):  
zuerst ändern, dann verwenden

- `++variable` für `variable = variable + 1`

- `--variable` für `variable = variable - 1`

- Veränderung mit beliebigen Wert (statt nur  $\pm 1$ )

- `variable += value` für `variable = variable + value;`

- `variable -= value` für `variable = variable - value;`

- `variable *= value` für `variable = variable * value;`

- `variable /= value` für `variable = variable / value;`

- `variable %= value` für `variable = variable % value;`

Achtung:

`x += 3`

`x =+ 3`

# **Coding mit Joran**

# Methoden: Folgen und Reihen

Schreiben Sie ein Programm “Reihe.java”, das eine Zahl  $N$  von der Konsole einliest und dann die folgende Summe berechnet:

$$\frac{1}{1^2} + \frac{1}{2^2} + \cdots + \frac{1}{N^2}$$

**Beispiel:** Für  $N = 4$  sollte Ihr Programm ca. 1.42 ausgeben. Wie verhält sich diese Summe für grosse  $N$ ?

# Cool Tricks in IntelliJ

- **psvm: Generate Main function**
- **sout: System.out.println()**
- **fori: For loop**
- **Ctrl + Alt + L / Cmd + Alt + L: Cleans up formatting**
- **Ctrl + Shift + Enter / Cmd + Shift + Enter: Finish Line**



# Strings

# Strings: Funktionalität

- Strings sind Objekte: Objekt `s` vom Typ `String`
  - Strings sind keine Arrays! Arrays folgen bald!
- Standardbibliothek enthält viel Funktionalität für Strings
  - Immer vorhanden, ohne `import java.util.*;`
  - Methoden werden mit dot-Notation aufgerufen: `s.method(parameters)`  
(führe Methode `method` für `s` aus, wende Methode `method` auf `s` an)
  - Ergebnis (Rückgabewert der Methode) kann sein: `String`, `int`, `boolean`, `char`
- Strings sind **unveränderbar** («**immutable**»)
  - Zuweisung verändert nicht den String: Variable verweist auf neuen String
  - Alle Methoden lassen String unverändert

# String-Methoden, die String liefern

| Name der Methode                                                   | Beschreibung der Methode                                                                                                                                                                 |
|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>substring(i1, i2)</code><br>or<br><code>substring(i1)</code> | Ein neuer String:<br>Der Substring von <code>i1</code> ( <b>inklusive</b> ) bis <code>i2</code> ( <b>exklusive</b> );<br>falls kein <code>i2</code> übergeben wird: bis Ende des Strings |
| <code>toLowerCase()</code>                                         | Ein neuer String mit nur Kleinbuchstaben                                                                                                                                                 |
| <code>toUpperCase()</code>                                         | Ein neuer String mit nur Grossbuchstaben                                                                                                                                                 |
| <code>stripLeading()</code>                                        | Ein neuer String ohne Leerzeichen am Anfang                                                                                                                                              |
| <code>stripTrailing()</code>                                       | Ein neuer String ohne Leerzeichen am Ende                                                                                                                                                |

# Substrings

```
String name = "Bob Dylan";  
String firstName = name.substring(0, 3); // "Bob"  
String lastName = name.substring(4); // "Dylan"  
String firstCharacter = name.substring(0, 1); // "B"  
name = name.toLowerCase(); // "bob dylan"
```

Indizes 0, 1, 2

Von Index 4  
bis Ende

|       |   |   |   |   |   |   |   |   |   |
|-------|---|---|---|---|---|---|---|---|---|
| name: | B | o | b |   | D | y | l | a | n |
| Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

Achtung: `name.substring(0, 1)` ist nicht das gleiche wie `name.charAt(0)`

- `name.substring(0, 1)` hat Wert "B" und ist vom Typ String
- `name.charAt(0)` hat Wert 'B' und ist vom Typ char

# String-Methoden, die `int` liefern

| Name der Methode                                                                                                               | Beschreibung der Methode                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>length()</code>                                                                                                          | Länge des Strings (Anzahl Zeichen)                                                                                                                                                                                                      |
| <code>indexOf(s, fromIndex)</code><br><code>indexOf(c, fromIndex)</code><br><code>indexOf(s)</code><br><code>indexOf(c)</code> | Index wo String <code>s</code> oder Zeichen <code>c</code> zum ersten Mal nach Index <code>fromIndex</code> im String auftaucht (-1 falls nicht gefunden);<br>falls <code>fromIndex</code> nicht übergeben wird, von Anfang des Strings |

```
String s = "S. Beckett";  
int indexBeck = s.indexOf("Beck");           // 3  
int eFirstOccurrence = s.indexOf('e');       // 4  
int eSecondOccurrence = s.indexOf('e', 5);    // 7
```

# String-Methoden, die boolean liefern

| Name der Methode                  | Beschreibung der Methode bei Aufruf <code>s1.method(s2)</code>                                       |
|-----------------------------------|------------------------------------------------------------------------------------------------------|
| <code>equals(s2)</code>           | ob s1 und s2 die gleichen Buchstaben enthalten                                                       |
| <code>equalsIgnoreCase(s2)</code> | ob s1 und s2 die gleichen Buchstaben enthalten, ohne Berücksichtigung von Gross- und Kleinschreibung |
| <code>startsWith(s2)</code>       | ob s1 mit den Buchstaben von s2 anfängt                                                              |
| <code>endsWith(s2)</code>         | ob s1 mit den Buchstaben von s2 endet                                                                |
| <code>contains(s2)</code>         | ob s1 irgendwo s2 enthält                                                                            |

Hinweis zu `==` und `equals`

- `==` nur für Basistypen (z.B. `int` oder `char`), nicht für Strings
- `==` für Strings wird später erklärt!

# Wo findet man diese Informationen?

## ■ In der Java Documentation 21

Java SE 21 & JDK 21

SEARCH

### Java® Platform, Standard Edition & Java Development Kit Version 21 API Specification

This document is divided into two sections:

**Java SE**

The Java Platform, Standard Edition (Java SE) APIs define the core Java platform for general-purpose computing. These APIs are in modules whose names start with `java`.

**JDK**

The Java Development Kit (JDK) APIs are specific to the JDK and will not necessarily be available in all implementations of the Java SE Platform. These APIs are in modules whose names start with `jdk`.

All ModulesJava SEJDKOther Modules

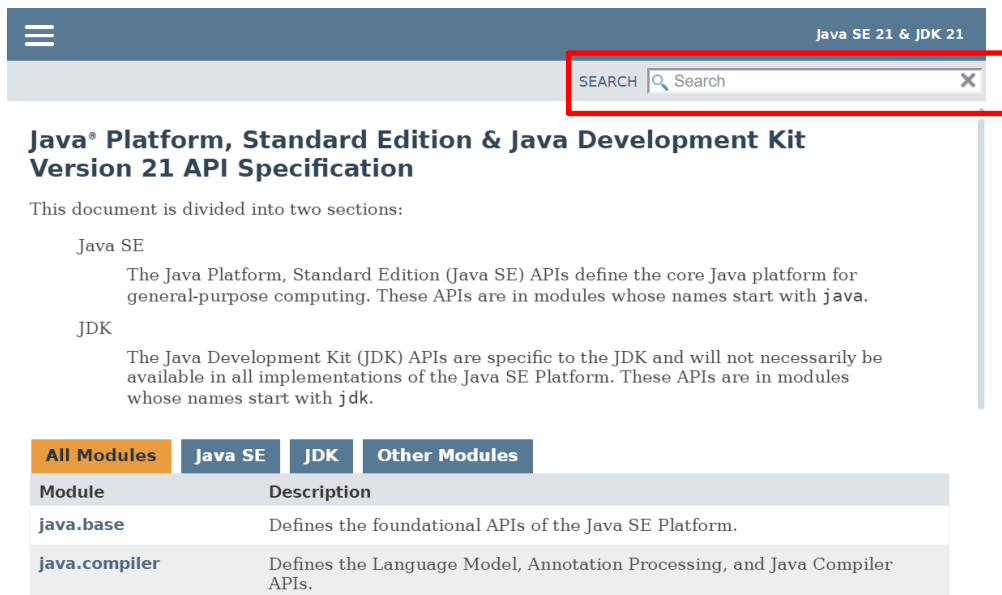
| Module                     | Description                                                                |
|----------------------------|----------------------------------------------------------------------------|
| <code>java.base</code>     | Defines the foundational APIs of the Java SE Platform.                     |
| <code>java.compiler</code> | Defines the Language Model, Annotation Processing, and Java Compiler APIs. |



<https://docs.oracle.com/en/java/javase/21/docs/api/>

# Wo findet man diese Informationen?

## ■ In der Java Documentation 21



Java SE 21 & JDK 21

SEARCH

### Java® Platform, Standard Edition & Java Development Kit Version 21 API Specification

This document is divided into two sections:

**Java SE**

The Java Platform, Standard Edition (Java SE) APIs define the core Java platform for general-purpose computing. These APIs are in modules whose names start with `java`.

**JDK**

The Java Development Kit (JDK) APIs are specific to the JDK and will not necessarily be available in all implementations of the Java SE Platform. These APIs are in modules whose names start with `jdk`.

| All Modules                | Java SE                                                                    | JDK | Other Modules |
|----------------------------|----------------------------------------------------------------------------|-----|---------------|
| Module                     | Description                                                                |     |               |
| <code>java.base</code>     | Defines the foundational APIs of the Java SE Platform.                     |     |               |
| <code>java.compiler</code> | Defines the Language Model, Annotation Processing, and Java Compiler APIs. |     |               |

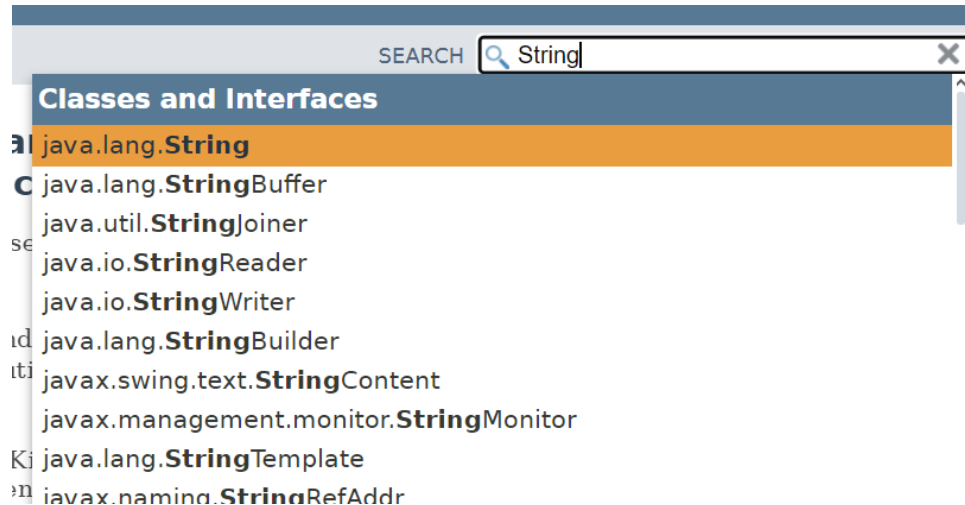


<https://docs.oracle.com/en/java/javase/21/docs/api/>



# Wo findet man diese Informationen?

- In der Java Documentation 21



<https://docs.oracle.com/en/java/javase/21/docs/api/>

# Wo findet man diese Informationen?

- In der Java Documentation 21



<https://docs.oracle.com/en/java/javase/21/docs/api/>

## Method Summary

All Methods

Static Methods

Instance Methods

Concrete Methods

Deprecated Methods

| Modifier and Type | Method                                                       | Description                                                                           |
|-------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------|
| char              | <a href="#">charAt(int index)</a>                            | Returns the char value at the specified index.                                        |
| IntStream         | <a href="#">chars()</a>                                      | Returns a stream of int zero-extending the char values from this sequence.            |
| int               | <a href="#">codePointBefore(int index)</a>                   | Returns the character (Unicode code point) before the specified index.                |
| int               | <a href="#">codePointCount(int beginIndex, int endIndex)</a> | Returns the number of Unicode code points in the specified text range of this String. |
| IntStream         | <a href="#">codePoints()</a>                                 | Returns a stream of code point values from this sequence.                             |
| int               | <a href="#">compareTo(String anotherString)</a>              | Compares two strings lexicographically.                                               |
| int               | <a href="#">compareToIgnoreCase(String str)</a>              | Compares two strings lexicographically, ignoring case differences.                    |
| String            | <a href="#">concat(String str)</a>                           | Concatenates the specified string to the end of this string.                          |

**Methodennamen anklicken**

# Wo findet man diese Informationen?

## ■ In der Java Documentation 21

### charAt

```
public char charAt(int index)
```

Returns the `char` value at the specified index. An index ranges from 0 to `length() - 1`. The first `char` value of the sequence is at index 0, the next at index 1, and so on, as for array indexing.

If the `char` value specified by the index is a [surrogate](#), the surrogate value is returned.

#### Specified by:

`charAt` in interface [CharSequence](#)

#### Parameters:

`index` - the index of the `char` value.

#### Returns:

the `char` value at the specified index of this string. The first `char` value is at index 0.

#### Throws:

[IndexOutOfBoundsException](#) - if the `index` argument is negative or not less than the length of this string.



# Escaping in Strings

- **Strings in Java sind gekennzeichnet durch `""` (reservierte Symbole)**
- **Was machen wir, wenn wir ein reserviertes Symbol im String verwenden wollen?**
- **Wir können es mit `\` (ebenfalls ein reserviertes Symbol) "escapen"**

|||||

## Aussengrenzen des Strings

### Escape-Charakter

### Im Resultat sichtbar

Diagram illustrating the visual representation of a string and its escape characters. The string is enclosed in double quotes. The characters within the string are: a backslash (red), a single quote (green), five backslashes (black), a double quote (black), and another backslash (black). The diagram shows how these characters are escaped in the result, making them visible.

Aussengrenzen des Strings  
Escape-Charakter  
Im Resultat sichtbar



Diagram illustrating the visual representation of a string and its escape characters. The string is enclosed in double quotes. Inside, there are single quotes, a backslash, and a double quote, each preceded by a backslash. The characters are color-coded: the outer quotes are purple, the inner single quotes are green, the backslashes are red, and the inner double quote is black.

Aussengrenzen des Strings  
Escape-Charakter  
Im Resultat sichtbar





Diagram illustrating the visual representation of a string and its escape sequences. The string is enclosed in double quotes, and the escape sequences (backslashes) are shown as diagonal lines. The characters are color-coded: purple for the string boundaries, red for the opening and closing quotes, green for the escaped quotes, and black for the escaped backslashes.

Aussengrenzen des Strings  
Escape-Charakter  
Im Resultat sichtbar



Aussengrenzen des Strings  
Escape-Charakter  
Im Resultat sichtbar

""\\""

Aussengrenzen des Strings  
Escape-Charakter  
Im Resultat sichtbar

# String Beispiele



```
1 public class MyClass{
2     public static void main(String[] args){
3         String str1 = "Java21";
4         String str2 = "EPROG2024";
5         String result = str1.substring(0, 2) + str2.substring(str2.length() - 4);
6     }
7 }
```

Output: Ja2024

| Name der Methode                         | Beschreibung der Methode                                                                                                                                               |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| substring(i1, i2)<br>or<br>substring(i1) | Ein <u>neuer</u> String:<br>Der Substring von i1 ( <b>inklusive</b> ) bis i2 ( <b>exklusive</b> );<br>falls <u>kein</u> i2 <u>übergeben</u> wird: bis Ende des Strings |

# String Beispiele



```
1 public class MyClass{  
2     public static void main(String[] args){  
3         String str = "Substring";  
4         int result = str.indexOf('S') + str.lastIndexOf('g') - str.length();  
5     }  
6 }
```

**Output: -1**

# String Beispiele



```
1 public class MyClass{
2     public static void main(String[] args){
3         String str = "Programmierung";
4         String result = str.charAt(0) + str.substring(2, 5).toLowerCase() + str.charAt(str.length() - 1);
5     }
6 }
```

Output: Pogrgr

# Kahoot

# **Vorbesprechung**



# Aufgabe 1: Binärdarstellung

Schreiben Sie ein Programm “Binaer.java”, das eine positive Zahl  $Z$  einliest und dann die Binärdarstellung druckt. (Hinweis: Finden Sie zuerst die grösste Zahl  $k$ , so dass die Zweierpotenz  $K = 2^k$  kleiner als  $Z$  ist.)

**Beispiel:** Für  $Z = 14$  sollte Ihr Programm 1110 ausgeben.

# Aufgabe 2: Grösster gemeinsamer Teiler

Schreiben Sie ein Programm “GGT.java”, das den grössten gemeinsamen Teiler (ggT) zweier ganzer Zahlen mithilfe des Euklidischen Algorithmus berechnet. Hierbei handelt es sich um eine iterative Berechnung, die auf folgender Beobachtung basiert:

1. Wenn  $x$  grösser oder gleich  $y$  ist und sich  $x$  durch  $y$  teilen lässt, dann ist der ggT von  $x$  und  $y$  gleich  $y$ ;
2. andernfalls ist der ggT von  $x$  und  $y$  der gleiche wie der ggT von  $y$  und  $x \% y$ .

Üben Sie diesen Algorithmus zuerst von Hand an ein paar Beispielen und schreiben Sie dann das Java Programm.

**Beispiel:** Für  $x = 36$  und  $y = 44$ :

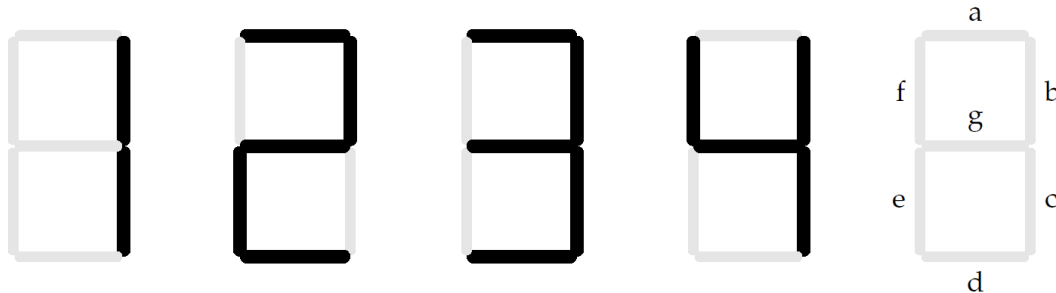
$$\text{ggT}(\underset{x}{36}, \underset{y}{44}) \stackrel{2.}{=} \text{ggT}(44, \underbrace{36 \% 44}_{36}) \stackrel{2.}{=} \text{ggT}(36, \underbrace{44 \% 36}_{8}) \stackrel{2.}{=} \text{ggT}(8, \underbrace{36 \% 8}_{4}) \stackrel{1.}{=} 4$$

# Aufgabe 3: Zahlenerkennung

Für diese Aufgabe verwenden wir einen String um die erleuchtenden Segmente einer [Sieben-segmentanzeige](#) zu kodieren. Die Segmente sind, wie im Bild gezeigt, von a bis g nummeriert. Die Kodierung einer möglichen Anzeige ist ein String, in welchem der Buchstabe 'x' genau dann vorkommt, wenn das 'x'te Segment der Anzeige erleuchtet ist. Zum Beispiel wird die Zahl 2 kodiert durch 'abged'. Zur Einfachheit darf angenommen werden, dass kein Buchstabe mehr als einmal in der Kodierung vorkommt und dass nur die Zahlen 0 bis 9 kodiert werden.

Schreiben Sie ein Programm "Zahlen.java", das einen String, der eine Anzeige kodiert, einliest und die kodierte Zahl als Integer ausgibt. Überlegen Sie wie viele IF Blöcke benötigt werden um jede Zahl zu erkennen.

**Tip:** Sie können `str.contains("a")` verwenden, um zu überprüfen, ob ein String `str` den Buchstaben 'a' enthält.



# Aufgabe 4: Berechnungen

2. Implementieren sie die Methode `Calculations.magic7(int a, int b)`. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn einer der Parameter 7 ist oder wenn die Summe oder Differenz der Parameter 7 ist. Ansonsten soll die Methode `false` zurückgeben.

Beispiele

- `magic7(2,5)` gibt `true` zurück.
- `magic7(7,9)` gibt `true` zurück.
- `magic7(5,6)` gibt `false` zurück.

Hinweis: Mit der Funktion `Math.abs(num)` können Sie den absoluten Wert einer Zahl `num` erhalten.

# Aufgabe 4: Berechnungen

3. Implementieren Sie die Methode `Calculations.fast12(int z)`. Das Argument `z` ist nicht negativ. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn `z` nahe an einem Vielfachen von 12 ist. Eine Zahl `x` ist nahe an einer Zahl `y`, wenn eine der Zahlen um maximal 2 grösser oder kleiner ist als die andere Zahl. Ansonsten soll die Methode `false` zurückgeben.
- `fast12(12)` gibt `true` zurück.
  - `fast12(14)` gibt `true` zurück.
  - `fast12(10)` gibt `true` zurück.
  - `fast12(15)` gibt `false` zurück.

# Aufgabe 5: Scrabble

In dieser Aufgabe sollen Sie Scrabble-Steine legen, mittels ASCII-Art auf der Konsole. Vervollständigen Sie die Methode `drawNameSquare` in der Klasse `Scrabble`. Diese Methode nimmt einen Namen als `String`-Parameter und soll den Namen als in einem Quadrat angeordnete Scrabble-Steine auf der Konsole (`System.out`) ausgeben. Wenn z.B. der `String` `Alfred` übergeben wird, sollte folgendes Bild ausgegeben werden:

```
+---+---+---+---+---+---+
| A | L | F | R | E | D |
+---+---+---+---+---+---+
| L |           | E |
+---+           +---+
| F |           | R |
+---+           +---+
| R |           | F |
+---+           +---+
| E |           | L |
+---+---+---+---+---+---+
| D | E | R | F | L | A |
+---+---+---+---+---+---+
```

# Aufgabe 5: Scrabble

Ihr Code braucht nur Namen der Länge 3 oder länger unterstützen. Für einen Namen mit Länge 3, z.B. Jim, sollte die Ausgabe so aussehen:

```
+---+---+---+  
| J | I | M |  
+---+---+---+  
| I |   | I |  
+---+---+---+  
| M | I | J |  
+---+---+---+
```

Beachten Sie, dass Ihr Programm keinerlei andere Ausgabe als das Scrabble-Quadrat machen darf.

# Prüfungsaufgabe FS22 – 3/40 Punkte

Gegeben sei eine Methode main in einer Java Klasse.

```
public static void main(String[] args) {  
    /* body */  
}
```

Die folgenden Anweisungen sollen als “Body” (Rumpf) anstelle des Kommentars `/* body */` eingefügt werden. Geben Sie für jede Anweisung an, was für eine Ausgabe erzeugt wird – entweder was gedruckt wird, oder ob ein Laufzeitfehler auftritt (schreiben Sie “Exception”), oder ob der Compiler einen Fehler feststellt (schreiben Sie “Compile-Fehler”). Achten Sie auf die korrekte Formatierung der verschiedenen Typen, also z.B. 7.0 statt 7 für eine reelle Zahl (double).



1. `System.out.println(11 + 16 / 4 * 2 + (4 + ">") + 4 * 2);`

**194>8**

---

2. `System.out.println(20 / 10 % 6 + 3 / 2 + (double) 5 / 4 + 3 / 4);`

**4.25**

---

3. `System.out.println( ( 11 % 4 ) > 2 && 9 > (16 / (2 / 4)) && 1 % 8 < 0);`

**Exception: Division by 0**

---